

ORACLE SQL

First-Half Exam Reviewer

In-depth capabilities, limitations, comparisons, and certification-style questions

Scope: This is the first half shown in the first syllabus image: relational database concepts; SELECT retrieval; restricting and sorting; single-row functions; conversion and conditional expressions; and aggregate reporting.

Study design: Every topic follows the same pattern: meaning -> capability -> syntax -> limitations -> comparison -> exam trap -> practice items.

How the Exam Constructs Questions

- A small table definition or data set is shown, followed by one statement and several alternatives that differ by one rule.
- Words such as can, must, always, only, required, successful, and fails are decisive.
- For alternative-answer items, none, some, or all options may be true. Evaluate each alternative independently.
- Oracle-specific behavior is frequently tested: NULL, implicit conversion, default NULL sorting, date arithmetic, aliases, grouping rules, and SQL*Plus substitution.
- A statement can be logically meaningful but syntactically invalid. Check syntax before predicting data.
- When output order is requested, compute every ORDER BY key, including CASE expressions and tie-breakers.

Seven-step method: 1) classify the SQL; 2) inspect data types; 3) check syntax; 4) apply FROM/WHERE/GROUP BY/HAVING/SELECT/ORDER BY; 5) track NULL; 6) calculate intermediate results; 7) test each option independently.

Module 1 - Relational Database Concepts

1.1 Logical versus physical database aspects

The relational model describes data logically as relations, while the database engine stores the data physically. SQL normally states the required result, not the physical route used to obtain it.

Capabilities

- Logical design defines entities, attributes, keys, relationships, constraints, and views.
- Physical storage includes data files, blocks, extents, segments, indexes, and storage parameters.
- Logical data independence allows SQL to remain stable when physical storage or access paths change.
- The optimizer decides access paths such as full table scan or index access.

Syntax and example

```
-- Logical request: no physical access method is specified
SELECT employee_id, employee_name
FROM employees
WHERE department_id = 20;
```

Limitations and conditions

- SQL result order is not guaranteed unless ORDER BY is present.
- An index may influence performance but does not change the logical result.
- A primary key is a logical integrity rule; its supporting index is a physical access structure and should not be treated as the same concept.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
Table	Represents a logical relation	A data file or physical block
Primary key	Identifies rows logically	ORDER BY or physical row order
Index	Provides an optional access path	A guarantee that Oracle must use it

Exam construction pattern: Choices often mix a logical rule with a physical implementation. A statement claiming that a primary key always controls output order is false.

1.2 ERD components and SELECT clauses

An ERD models entities, attributes, identifiers, and relationships. SQL clauses use the resulting tables and columns to retrieve data.

Capabilities

- Entity maps to a table; attribute maps to a column; entity occurrence maps to a row.
- Primary identifier maps to a primary or unique key.
- Relationship usually maps to a foreign key.

- FROM selects entity sources; SELECT chooses attributes; JOIN/ON expresses relationships; WHERE applies row criteria.

Syntax and example

```
SELECT e.employee_name, d.department_name
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id
WHERE d.location_id = 1700;
```

Limitations and conditions

- A join condition retrieves related rows but does not itself create the schema constraint.
- A foreign key enforces valid relationships for stored data but does not automatically cause every SELECT to join tables.
- A many-to-many relationship normally requires an associative table.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
SELECT list	Attributes to display	Rows to remove
FROM/JOIN	Entity sources and query relationships	Stored primary-key definition
WHERE	Row restriction	Group restriction after aggregation

Exam construction pattern: Questions may ask which SQL clause corresponds to an ERD component. Carefully distinguish a stored relationship from query-time joining.

1.3 Relationship between a database and SQL

The database stores and manages structured data and metadata. SQL is the language used to communicate declarative requests to that database.

Capabilities

- DQL retrieves data with SELECT.
- DML changes rows with INSERT, UPDATE, DELETE, and MERGE.
- DDL manages objects with CREATE, ALTER, DROP, TRUNCATE, and RENAME.
- DCL manages privileges with GRANT and REVOKE; TCL manages transactions with COMMIT, ROLLBACK, and SAVEPOINT.

Syntax and example

```
SELECT * FROM employees;           -- retrieval
UPDATE employees SET salary=salary*1.05; -- row change
CREATE TABLE demo(id NUMBER);    -- object definition
GRANT SELECT ON demo TO reporter;  -- access
COMMIT;                          -- transaction control
```

Limitations and conditions

- SQL is declarative: it states what result is wanted, while Oracle chooses how to execute it.
- DDL and DML are not interchangeable.
- A valid SQL statement still requires privileges on referenced objects.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
SELECT	Read rows	DDL object creation

UPDATE	Change existing rows	ALTER TABLE structure change
COMMIT	Make transaction work permanent	SAVEPOINT marker

Exam construction pattern: Distractors often use a real command from the wrong command family.

Practice Item 1

Which statement is true about a relational database?

- A. A table guarantees that rows are returned in insertion order.
- B. Logical table design is independent of the exact physical access path used for a query.
- C. Every foreign key must contain a non-NULL value.
- D. An index and a primary-key constraint are identical concepts.

Correct answer: B

Why

SQL operates on the logical model, while Oracle can change the physical access path without changing the requested result.

Why the alternatives differ

- A. Incorrect. Only ORDER BY guarantees result ordering.
- B. Correct. Logical requests are separated from physical access decisions.
- C. Incorrect. A foreign key can be nullable unless NOT NULL is also declared.
- D. Incorrect. A key is an integrity rule; an index is an access structure.

Rule to remember: Separate logical rules from physical implementation.

Practice Item 2

Which SQL component most directly expresses the relationship between EMPLOYEES and DEPARTMENTS during retrieval?

- A. SELECT
- B. JOIN ... ON
- C. ORDER BY
- D. DISTINCT

Correct answer: B

Why

JOIN ... ON supplies the query-time relationship condition connecting rows from the two table sources.

Why the alternatives differ

- A. SELECT chooses output expressions.
- B. Correct. ON describes how rows match.
- C. ORDER BY sorts the final result.

D. DISTINCT removes duplicate result rows.

Rule to remember: ERD relationship -> usually foreign key in design and join condition in retrieval.

Module 2 - Retrieving Data with SELECT

2.1 SELECT statement and column aliases

SELECT retrieves expressions from one or more row sources. An alias changes an output heading, not the stored column name.

Capabilities

- Select all columns with * or list only required expressions.
- Use AS optionally for column aliases.
- Use double quotes when an alias contains spaces, mixed case, or special characters.
- Aliases improve readability and can be referenced by ORDER BY in ordinary queries.

Syntax and example

```
SELECT employee_id,
       salary * 12 AS annual_salary,
       first_name || ' ' || last_name AS "Employee Name"
FROM employees;
```

Limitations and conditions

- A SELECT expression produces one result column even when it contains several source columns.
- A column alias normally cannot be used in the same SELECT block WHERE clause because WHERE is logically evaluated before SELECT.
- Single quotes create character literals; they do not correctly delimit ordinary column aliases.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
salary annual_salary	Simple alias	Alias containing a space
salary AS annual_salary	Explicit readable alias	A stored column rename
salary AS "Annual Salary"	Alias with a space/case	A string literal

Exam construction pattern: Common choices differ only by single versus double quotes or by attempting to use the alias in WHERE.

2.2 Concatenation, literals, alternative quoting, and DISTINCT

Concatenation builds text from expressions. Literals add fixed values. DISTINCT removes duplicate complete result rows.

Capabilities

- || concatenates values in Oracle SQL.
- Character and date-like text literals use single quotes.
- Alternative quoting q'[text]' allows embedded apostrophes more readably.
- DISTINCT applies to all selected expressions as one tuple.

Syntax and example

```
SELECT employee_id,
       first_name || ' ' || last_name || ' earns ' || salary AS summary,
```

```
q'[Employee's profile]' AS label
FROM employees;
```

```
SELECT DISTINCT department_id, job_id
FROM employees;
```

Limitations and conditions

- Additional spaces must be included as literals; concatenation does not insert them automatically.
- DISTINCT is written once after SELECT, not before each column.
- DISTINCT does not guarantee sorted output.
- Concatenating a NULL in Oracle can still return the non-NULL text portions; do not assume the entire string always becomes NULL as arithmetic does.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
DISTINCT a	Unique a values	ORDER BY a
DISTINCT a,b	Unique (a,b) combinations	Distinct a independent of b
q'[Employee's row]'	Alternative quote literal	Double-quoted identifier

Exam construction pattern: The exam may provide repeated values and ask how many distinct combinations result.

2.3 Arithmetic expressions and NULL

Arithmetic expressions calculate values for each row. NULL represents unknown or missing information, not zero.

Capabilities

- Multiplication and division precede addition and subtraction.
- Parentheses override arithmetic precedence.
- Arithmetic with NULL normally returns NULL.
- Use NVL or COALESCE only when the business rule genuinely treats missing data as a replacement value.

Syntax and example

```
SELECT salary,
       salary * 12 annual_salary,
       salary + salary * NVL(commission_pct,0) total_pay
FROM employees;
```

Limitations and conditions

- NULL cannot be tested with = or <>.
- Replacing NULL with zero changes business meaning and should be intentional.
- Division by zero raises an error; NULLIF(denominator,0) can turn zero into NULL when that result is acceptable.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
salary + commission	Adds two values	Treating NULL as zero
NVL(commission,0)	Explicit NULL replacement	Testing IS NULL
NULLIF(quantity,0)	Returns NULL when quantity is zero	NVL fallback

Exam construction pattern: Questions commonly give one NULL operand and offer zero, unchanged salary, NULL, and error as choices.

Practice Item 3

Which statement correctly creates the heading Annual Salary with a space?

- A. SELECT salary AS 'Annual Salary' FROM employees;
- B. SELECT salary AS "Annual Salary" FROM employees;
- C. SELECT salary Annual Salary FROM employees;
- D. SELECT salary = "Annual Salary" FROM employees;

Correct answer: B

Why

Double quotation marks delimit an identifier containing a space.

Why the alternatives differ

- A. Single quotes delimit text literals.
- B. Correct.
- C. ANNUAL may be read as the alias and SALARY becomes unexpected syntax.
- D. An alias does not use the equality operator.

Rule to remember: Use double quotes for spaced aliases.

Practice Item 4

SELECT DISTINCT department_id, job_id applies DISTINCT to:

- A. department_id only
- B. job_id only
- C. the combination of department_id and job_id
- D. each expression separately, producing two result sets

Correct answer: C

Why

Duplicate elimination considers the entire result row.

Why the alternatives differ

- A. Incorrect.
- B. Incorrect.
- C. Correct.
- D. One SELECT produces one result set.

Rule to remember: DISTINCT applies to the complete SELECT list.

Practice Item 5

SALARY is 5000 and COMMISSION_PCT is NULL. What does salary + salary * commission_pct return?

- A. 5000
- B. 0
- C. NULL
- D. A syntax error

Correct answer: C

Why

Multiplication by NULL returns NULL, and adding NULL returns NULL.

Why the alternatives differ

- A. Would require NVL(commission_pct,0).
- B. NULL is not zero.
- C. Correct.
- D. The expression is syntactically valid.

Rule to remember: Track NULL before performing arithmetic.

Module 3 - Restricting and Sorting Data

3.1 Comparison conditions and logical precedence

WHERE retains rows whose condition evaluates to TRUE. FALSE and UNKNOWN rows are excluded.

Capabilities

- Comparison: =, <>, !=, >, <, >=, <=.
- Range: BETWEEN; membership: IN; pattern: LIKE; missing value: IS NULL.
- Logical precedence: NOT, then AND, then OR.
- Parentheses make intended business logic explicit.

Syntax and example

```
SELECT employee_name
FROM employees
WHERE (department_id IN (10,20))
AND salary >= 5000
AND employee_name LIKE '_A%';
```

Limitations and conditions

- BETWEEN includes both endpoints.
- NOT IN can be affected by NULL in its list or subquery.
- LIKE _ matches exactly one character; % matches zero or more.
- = NULL and <> NULL evaluate UNKNOWN, not TRUE.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
BETWEEN 5 AND 10	Inclusive continuous range	IN (5,10), which tests only two values
IN (10,20)	Fixed membership list	BETWEEN 10 AND 20
IS NULL	Missing-value test	= NULL

Exam construction pattern: Options commonly differ by AND/OR grouping, inclusive endpoints, or wildcard position.

3.2 Limiting rows

Row limiting returns only part of an ordered or unordered result. The exam tests whether the selected rows are deterministic.

Capabilities

- FETCH FIRST n ROWS ONLY returns at most n rows.
- OFFSET m ROWS skips rows before fetching.
- ORDER BY should precede row limiting when top or bottom business meaning is intended.
- WITH TIES may return more than n rows when the final ORDER BY value ties, if supported by the used syntax/version.

Syntax and example

```
SELECT employee_id, salary
FROM employees
ORDER BY salary DESC, employee_id
OFFSET 0 ROWS FETCH FIRST 5 ROWS ONLY;
```

Limitations and conditions

- Without ORDER BY, which rows are first is not guaranteed.
- ORDER BY only salary may leave equal-salary ordering nondeterministic.
- ROWNUM top-N patterns have different placement rules; do not assume ORDER BY and ROWNUM in one block produce post-sort limiting.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
FETCH FIRST 5 ROWS ONLY	Modern row limiting	Guaranteeing highest values without ORDER BY
OFFSET 5 ROWS	Skip first five final rows	Deleting five rows
ROWNUM <= 5	Older limiting pattern	Always equivalent placement with ORDER BY

Exam construction pattern: The key distractor is a query that limits rows before establishing the required sort.

3.3 Substitution variables, DEFINE, VERIFY

These commands are client-side features used by SQL*Plus-style tools, not database table objects.

Capabilities

- &name substitutes a value and prompts when undefined.
- &&name defines the value for reuse after the first prompt.
- DEFINE name=value predefines text.
- UNDEFINE name removes it.
- SET VERIFY ON displays old and new statement text.

Syntax and example

```
DEFINE dept = 20
SET VERIFY ON
SELECT employee_name
FROM employees
WHERE department_id = &dept;
UNDEFINE dept
```

Limitations and conditions

- Text substitution happens before SQL parsing.
- A character value usually requires quotes around the substitution reference.
- Substitution variables are not bind variables and do not provide the same parsing/security characteristics.
- VERIFY displays replacement; it does not validate table data.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
&dept	Use or prompt for text	Bind variable :dept
&&dept	Define and reuse	Two unrelated prompts

SET VERIFY ON	Show before/after substitution	Check constraint validation
---------------	--------------------------------	-----------------------------

Exam construction pattern: Questions often ask which symbol prevents repeated prompting or what VERIFY actually displays.

3.4 ORDER BY

ORDER BY sorts the final result using columns, expressions, aliases, or positions where permitted.

Capabilities

- ASC is default; DESC reverses ordering.
- Oracle defaults NULLS LAST for ASC and NULLS FIRST for DESC.
- Multiple sort expressions are applied left to right.
- A later key breaks ties left by earlier keys.

Syntax and example

```
SELECT employee_id, employee_name, manager_id
FROM employees
ORDER BY manager_id DESC NULLS LAST,
       employee_id ASC;
```

Limitations and conditions

- Position-based ordering is fragile when the SELECT list changes.
- DISTINCT and set operations can impose restrictions on ORDER BY expressions.
- Without a tie-breaker, tied rows may appear in any relative order.
- ORDER BY alias is generally valid; WHERE alias is generally not in the same query block.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
ORDER BY salary DESC	Highest values first	NULL placement explicitly controlled
... DESC NULLS LAST	Highest non-NULL first, NULL last	Default DESC NULL placement
ORDER BY 2	Second SELECT item	Column named 2

Exam construction pattern: Exam questions frequently provide four rows with NULL and duplicate sort keys. Compute primary and secondary keys manually.

Practice Item 6

What does department_id=10 OR department_id=20 AND salary>5000 mean without parentheses?

- A. Both departments require salary>5000.
- B. All department 10 rows plus department 20 rows above 5000.
- C. Only department 20 rows.
- D. Invalid syntax.

Correct answer: B

Why

AND is evaluated before OR.

Why the alternatives differ

- A. Would require parentheses around the OR condition.
- B. Correct.
- C. Department 10 can qualify.
- D. Parentheses are recommended but not required.

Rule to remember: NOT, AND, OR precedence matters.

Practice Item 7

Which statement about BETWEEN 5000 AND 10000 is true?

- A. Both endpoints are excluded.
- B. Only 5000 is included.
- C. Both endpoints are included.
- D. It is equivalent to IN (5000,10000).

Correct answer: C

Why

BETWEEN is inclusive.

Why the alternatives differ

- A. Incorrect.
- B. Incorrect.
- C. Correct.
- D. IN tests only listed values.

Rule to remember: BETWEEN low AND high means $\geq$ low and $\leq$ high.

Practice Item 8

Which variable syntax prompts once and then keeps the substituted value defined?

- A. &dept
- B. &&dept
- C. :dept
- D. VERIFY dept

Correct answer: B

Why

Double ampersand defines and reuses the substitution variable.

Why the alternatives differ

- A. May prompt again when still undefined.
- B. Correct.
- C. Colon denotes a bind variable in applicable clients.
- D. VERIFY is a display setting.

Rule to remember: Do not confuse substitution and bind variables.

Practice Item 9

For ORDER BY manager_id DESC with no NULLS clause, Oracle places NULL:

- A. First
- B. Last
- C. Randomly
- D. The query fails

Correct answer: A

Why

The Oracle default for descending order is NULLS FIRST.

Why the alternatives differ

- A. Correct.
- B. Would require NULLS LAST.
- C. The NULL placement default is defined.
- D. NULL sorting is valid.

Rule to remember: ASC -> NULLS LAST; DESC -> NULLS FIRST by default.

Practice Item 10

A top-five highest-salary query is most deterministic when it uses:

- A. FETCH FIRST 5 ROWS ONLY without ORDER BY
- B. ORDER BY salary DESC, employee_id then FETCH FIRST 5 ROWS ONLY
- C. DISTINCT only
- D. WHERE ROWNUM=5 only

Correct answer: B

Why

Ordering by salary and a tie-breaker defines which rows belong to the top five.

Why the alternatives differ

- A. Which five is not guaranteed.
- B. Correct.
- C. DISTINCT does not establish top order.
- D. It does not express a top-five sorted result.

Rule to remember: Sort first in logical intent, then limit.

Module 4 - Single-Row Functions

4.1 Character functions in SELECT and WHERE

Character functions transform or inspect text one row at a time.

Capabilities

- UPPER/LOWER/INITCAP change case.
- LENGTH counts characters; INSTR finds a substring position.
- SUBSTR extracts characters.
- TRIM removes specified edge characters; REPLACE substitutes occurrences.
- LPAD/RPAD extend display text to a target length.

Syntax and example

```
SELECT UPPER(last_name),
       LENGTH(last_name),
       SUBSTR(last_name,1,3),
       INSTR(last_name,'A')
FROM employees
WHERE UPPER(last_name) = 'SMITH';
```

Limitations and conditions

- SUBSTR starting positions are 1-based; negative positions count from the end.
- A function on a filtered column can prevent use of a normal index unless an appropriate function-based index exists.
- LENGTH includes spaces stored in the value; CHAR padding behavior can differ from VARCHAR2.
- TRIM is not a general all-position replacement.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
SUBSTR	Extract part by position	INSTR, which returns a position
TRIM	Remove edge characters	REPLACE anywhere
UPPER(col)	Case-normalized expression	Changing the stored value automatically

Exam construction pattern: A common item asks for the exact output or whether a normal index can support a function-wrapped predicate.

4.2 Number functions: ROUND, TRUNC, MOD

Numeric functions transform numbers without changing stored table data unless used in DML.

Capabilities

- ROUND(n,d) rounds to d decimal places; negative d rounds left of decimal.
- TRUNC(n,d) cuts off beyond d places.
- MOD(m,n) returns the remainder of m divided by n.

Syntax and example

```
SELECT ROUND(45.926,2),
       TRUNC(45.926,2),
       ROUND(145,-2),
```



```
MOD(11,4)
FROM dual;
```

Limitations and conditions

- ROUND and TRUNC with omitted precision operate at zero decimal places.
- Negative precision affects tens/hundreds positions.
- MOD is not the same as division quotient.
- Function results in SELECT do not modify stored values.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
ROUND(45.926,2)	45.93	TRUNC result 45.92
TRUNC(45.926,2)	45.92	ROUND behavior
MOD(11,4)	3 remainder	11/4 quotient

Exam construction pattern: The answer choices often differ by one digit or confuse negative precision.

4.3 Date arithmetic and date functions

Oracle DATE stores date and time to seconds but no time-zone region. Date arithmetic uses days.

Capabilities

- DATE + number adds days; DATE - DATE returns a number of days.
- ADD_MONTHS handles calendar months.
- MONTHS_BETWEEN returns month difference, possibly fractional.
- NEXT_DAY finds the next named weekday. LAST_DAY finds month end.
- TRUNC(date) removes lower time/date components according to format; ROUND(date) rounds.

Syntax and example

```
SELECT hire_date,
       hire_date + 7,
       ADD_MONTHS(hire_date,6),
       LAST_DAY(hire_date),
       TRUNC(hire_date)
FROM employees;
```

Limitations and conditions

- Adding 1 means one day, not one month.
- DATE contains time but not a time-zone component.
- Implicit character-to-date conversion depends on session format.
- ADD_MONTHS handles month length according to calendar rules; do not replace it with a fixed day count.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
date + 7	Seven days later	Seven months later
ADD_MONTHS(date,1)	Calendar month later	date + 30
TRUNC(date)	Remove time for default day precision	TO_CHAR display formatting

Exam construction pattern: Exam questions compare DATE+30 with ADD_MONTHS(date,1),

which can differ around month boundaries.

Practice Item 11

What does INSTR('DATABASE','A') return?

- A. The substring A
- B. The position of the first A
- C. The length of DATABASE
- D. A Boolean value

Correct answer: B

Why

INSTR returns a numeric character position.

Why the alternatives differ

- A. SUBSTR returns text.
- B. Correct.
- C. LENGTH returns length.
- D. INSTR returns a number.

Rule to remember: INSTR locates; SUBSTR extracts.

Practice Item 12

Which result pair is correct for 45.926 with precision 2?

- A. ROUND 45.92, TRUNC 45.92
- B. ROUND 45.93, TRUNC 45.92
- C. ROUND 45.93, TRUNC 45.93
- D. ROUND 46, TRUNC 45

Correct answer: B

Why

ROUND examines the next digit; TRUNC removes excess digits.

Why the alternatives differ

- A. ROUND is wrong.
- B. Correct.
- C. TRUNC is wrong.
- D. Precision 2 retains two decimals.

Rule to remember: Round versus cut off.

Practice Item 13

What does hire_date + 7 calculate?

- A. Seven months later
- B. Seven days later

- C. Seven hours later
- D. Invalid date arithmetic

Correct answer: B

Why

A whole number added to DATE represents days.

Why the alternatives differ

- A. Use ADD_MONTHS.
- B. Correct.
- C. Use 7/24 for seven hours.
- D. The operation is valid.

Rule to remember: Oracle DATE arithmetic uses days.

Practice Item 14

Which expression is most suitable for the same calendar day six months later?

- A. hire_date + 180 always
- B. ADD_MONTHS(hire_date,6)
- C. MONTHS_BETWEEN(hire_date,6)
- D. MOD(hire_date,6)

Correct answer: B

Why

ADD_MONTHS implements calendar-month movement.

Why the alternatives differ

- A. Fixed days do not equal six calendar months consistently.
- B. Correct.
- C. MONTHS_BETWEEN compares two dates.
- D. MOD is numeric remainder.

Rule to remember: Choose date functions for calendar semantics.

Practice Item 15

A normal index exists on LAST_NAME. Which predicate may require a function-based index for direct matching?

- A. last_name='SMITH'
- B. UPPER(last_name)='SMITH'
- C. last_name IS NULL
- D. last_name BETWEEN 'A' AND 'Z'

Correct answer: B

Why

The predicate searches the result of UPPER(last_name), not the raw indexed expression.

Why the alternatives differ

- A.** Directly matches the indexed column.
- B.** Correct.
- C.** Different NULL indexing considerations apply, but no function wraps the column.
- D.** Uses the raw column.

Rule to remember: Function-wrapped predicates and normal indexes are not automatically the same access expression.

Module 5 - Conversion Functions and Conditional Expressions

5.1 Implicit versus explicit conversion

Oracle can convert compatible data types implicitly, but explicit conversion is clearer and less dependent on settings.

Capabilities

- TO_CHAR converts dates or numbers to formatted text.
- TO_DATE converts text to DATE using a format model.
- TO_NUMBER converts formatted text to NUMBER.
- Data type precedence can cause Oracle to convert one operand to another type.

Syntax and example

```
SELECT TO_DATE('2026-09-10','YYYY-MM-DD'),
       TO_CHAR(hire_date,'DD-MON-YYYY'),
       TO_NUMBER('1,250.50','9,999.99')
FROM dual;
```

Limitations and conditions

- The format model must match the text.
- Implicit date conversion depends on NLS_DATE_FORMAT and possibly NLS_DATE_LANGUAGE.
- Comparing a NUMBER column to nonnumeric text can cause conversion errors.
- Applying TO_CHAR to an indexed NUMBER column changes the searched expression.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
TO_DATE	Text -> DATE	TO_CHAR DATE -> text
TO_NUMBER	Text -> number	Implicit numeric conversion
CAST	General supported type conversion	Format-model-specific conversion

Exam construction pattern: Choices may all parse under one session, but only the explicit matching format is reliable across sessions.

5.2 NVL, NULLIF, and COALESCE

These functions handle missing or exceptional values in different ways.

Capabilities

- NVL(expr,replacement) returns replacement only when expr is NULL.
- NULLIF(a,b) returns NULL when a equals b; otherwise a.
- COALESCE(a,b,c,...) returns the first non-NULL expression.
- NULLIF is useful for turning a zero denominator into NULL when a NULL quotient is acceptable.

Syntax and example

```
SELECT NVL(commission_pct,0),
       NULLIF(quantity,0),
       COALESCE(work_phone,mobile_phone,'N/A'),
       amount / NULLIF(quantity,0)
FROM orders;
```

Limitations and conditions

- Arguments must be type-compatible according to conversion rules.
- NVL takes two arguments; COALESCE accepts multiple alternatives.
- NULLIF compares values; it is not a general replacement function.
- Neither function changes stored data unless used in DML.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
NVL(a,b)	Two-value NULL replacement	Multiple-level fallback
COALESCE(a,b,c)	First non-NULL among many	Equality comparison
NULLIF(a,b)	NULL when equal	NVL replacement

Exam construction pattern: The test may ask which function prevents division by zero or which accepts several fallback columns.

5.3 Nested functions and conditional logic

Nested functions feed the result of an inner function into an outer function. CASE selects one result based on conditions.

Capabilities

- Oracle evaluates nested function calls from the innermost expression outward.
- Searched CASE tests Boolean conditions in order.
- Simple CASE compares one expression with candidate values.
- CASE returns one result, stopping at the first match.

Syntax and example

```
SELECT UPPER(SUBSTR(last_name,1,3)),
       ROUND(MONTHS_BETWEEN(SYSDATE,hire_date),0),
       CASE
         WHEN salary >= 10000 THEN 'HIGH'
         WHEN salary >= 5000 THEN 'MEDIUM'
         ELSE 'LOW'
       END salary_band
FROM employees;
```

Limitations and conditions

- Result branches should have compatible data types.
- Ordering of overlapping WHEN conditions changes the result.
- An alias created by CASE cannot generally be used in WHERE of the same query block.
- Deep nesting can be valid but harder to maintain and debug.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
Simple CASE	Compare one expression to values	Independent Boolean conditions
Searched CASE	Evaluate conditions in order	Returning multiple results

Nested functions	Transform intermediate output	Changing stored values automatically
------------------	-------------------------------	--------------------------------------

Exam construction pattern: The exam often uses overlapping salary thresholds to verify that CASE stops at the first matching branch.

Practice Item 16

Which comparison is least dependent on session date format?

- A. hire_date='10-SEP-2026'
- B. hire_date=TO_DATE('2026-09-10','YYYY-MM-DD')
- C. TO_CHAR(hire_date)='10-SEP-2026'
- D. hire_date='09/10/26'

Correct answer: B

Why

The input text and matching format model are explicit.

Why the alternatives differ

- A. Relies on implicit conversion.
- B. Correct.
- C. TO_CHAR without a model uses defaults.
- D. Ambiguous and implicit.

Rule to remember: Explicit matching conversion is exam-safe.

Practice Item 17

What does COALESCE(a,b,c,0) return?

- A. The largest value
- B. The first non-NULL value from left to right
- C. Zero always
- D. The first NULL value

Correct answer: B

Why

COALESCE selects the first non-NULL expression.

Why the alternatives differ

- A. It is not MAX.
- B. Correct.
- C. Zero is only the final fallback.
- D. It searches for non-NULL.

Rule to remember: COALESCE is a multi-level fallback.

Practice Item 18

What does NULLIF(10,10) return?

- A. 10
- B. 0
- C. NULL
- D. An error

Correct answer: C

Why

Equal arguments cause NULLIF to return NULL.

Why the alternatives differ

- A. Returned when arguments differ.
- B. No automatic zero.
- C. Correct.
- D. Valid numeric arguments.

Rule to remember: NULLIF means NULL when equal.

Practice Item 19

A CASE checks salary >= 5000 before salary >= 10000. What is the risk for salary 12000?

- A. It returns the first label and never reaches the higher threshold.
- B. It returns both labels.
- C. It raises an overlapping-condition error.
- D. It returns NULL automatically.

Correct answer: A

Why

CASE stops at the first true condition, so broad conditions placed first can hide specific ones.

Why the alternatives differ

- A. Correct.
- B. CASE returns one result.
- C. Overlap is allowed.
- D. A matching branch exists.

Rule to remember: Order overlapping CASE conditions carefully.

Practice Item 20

Which function is designed for multiple fallback expressions?

- A. NVL
- B. COALESCE
- C. NULLIF
- D. MOD

Correct answer: B

Why

COALESCE accepts a list and returns the first non-NULL value.

Why the alternatives differ

- A.** NVL has two arguments.
- B.** Correct.
- C.** NULLIF compares two values.
- D.** MOD returns a remainder.

Rule to remember: Compare by capability, not by similar NULL-related names.

Module 6 - Reporting Aggregated Data with Group Functions

6.1 Group functions

Aggregate functions return one value for a set of rows or one value per group.

Capabilities

- COUNT(*) counts rows. COUNT(expr) counts non-NULL expression values.
- SUM and AVG apply to numeric expressions.
- MIN and MAX work with comparable data types, including dates and text under applicable comparison rules.
- DISTINCT inside an aggregate removes duplicate argument values before aggregation.

Syntax and example

```
SELECT COUNT(*) row_count,
       COUNT(commission_pct) populated_commissions,
       AVG(salary) average_salary,
       COUNT(DISTINCT department_id) department_count
FROM employees;
```

Limitations and conditions

- All standard aggregates except COUNT(*) and certain special counting/grouping functions ignore NULL arguments.
- AVG(expr) uses the count of non-NULL values, not total rows.
- An aggregate without GROUP BY treats all qualifying rows as one group.
- COUNT never returns NULL; it returns zero when no rows qualify.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
COUNT(*)	Count rows	COUNT(column), which ignores NULL
COUNT(DISTINCT x)	Count unique non-NULL x	SELECT DISTINCT whole rows
AVG(x)	Mean of non-NULL x	SUM(x)/COUNT(*) when x contains NULL

Exam construction pattern: Questions often provide four rows with two NULLs and ask for COUNT and AVG results.

6.2 Creating groups with GROUP BY

GROUP BY divides qualifying rows into groups based on one or more expressions.

Capabilities

- One result row is produced per grouping combination when aggregate results are selected.
- Every nonaggregate SELECT expression must correspond to the grouping level.
- Grouping by multiple columns creates more granular groups.
- GROUP BY can use expressions, but the SELECT expression must be compatible with the grouping expression.

Syntax and example

```
SELECT department_id, job_id,
       COUNT(*) employee_count,
       AVG(salary) average_salary
FROM employees
GROUP BY department_id, job_id;
```

Limitations and conditions

- Oracle does not choose an arbitrary ungrouped employee name.
- Adding a column to GROUP BY can increase the number of result rows.
- GROUP BY does not guarantee sort order.
- A SELECT alias is generally not used as the GROUP BY expression in the same query block in traditional Oracle patterns; repeat the expression when required.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
GROUP BY department_id	One row per department	ORDER BY department_id
GROUP BY dept,job	One row per pair	Two independent summaries
DISTINCT dept	Remove duplicate dept outputs	Calculate aggregates

Exam construction pattern: A common invalid query selects department_id, employee_name, COUNT(*) while grouping only by department_id.

6.3 Restricting groups with HAVING

HAVING filters completed groups based on aggregate or group-level conditions.

Capabilities

- WHERE reduces input rows before GROUP BY.
- HAVING removes groups after aggregate values are available.
- HAVING can reference aggregate expressions.
- HAVING can be used without an explicit GROUP BY when the query forms one implicit group, if the condition is meaningful and valid.

Syntax and example

```
SELECT department_id, AVG(salary) avg_salary
FROM employees
WHERE status='ACTIVE'
GROUP BY department_id
HAVING AVG(salary) > 5000
ORDER BY avg_salary DESC;
```

Limitations and conditions

- HAVING is not required whenever GROUP BY appears.
- HAVING does not always reduce the result; its condition may be true for all groups.
- Row-level conditions usually belong in WHERE for clarity and earlier filtering.
- Aggregate functions cannot be used in WHERE to filter group results.

Compared with similar syntax

Syntax	Use it when	Do not confuse it with
WHERE salary>5000	Remove individual rows before grouping	Department average condition
HAVING AVG(salary)>5000	Remove groups after averaging	Source-row filter

GROUP BY	Create groups	Automatically restrict groups
----------	---------------	-------------------------------

Exam construction pattern: Sample questions frequently use can versus always. “HAVING can restrict groups” is true; “HAVING always restricts groups” is false.

Practice Item 21

Four commission values are 500, NULL, 700, NULL. What are COUNT(*), COUNT(commission), and AVG(commission)?

- A. 4,4,300
- B. 4,2,600
- C. 2,2,600
- D. 4,2,300

Correct answer: B

Why

COUNT(*) counts four rows; COUNT(column) counts two values; AVG is 1200/2.

Why the alternatives differ

- A. NULLs are not counted by COUNT(column).
- B. Correct.
- C. COUNT(*) is four.
- D. AVG does not divide by all four rows.

Rule to remember: Most aggregates ignore NULL.

Practice Item 22

Which query is invalid?

- A. SELECT department_id,COUNT(*) FROM employees GROUP BY department_id
- B. SELECT department_id,employee_name,COUNT(*) FROM employees GROUP BY department_id
- C. SELECT COUNT(*) FROM employees
- D. SELECT MAX(hire_date) FROM employees

Correct answer: B

Why

EMPLOYEE_NAME is neither grouped nor aggregated.

Why the alternatives differ

- A. Valid.
- B. Correct choice: invalid query.
- C. Valid one-group aggregate.
- D. MIN/MAX can operate on dates.

Rule to remember: Selected detail must match group grain.

Practice Item 23

Where should `AVG(salary)>5000` be written to restrict grouped departments?

- A. WHERE
- B. HAVING
- C. FROM
- D. DISTINCT

Correct answer: B

Why

The average exists only after groups are formed.

Why the alternatives differ

- A. WHERE is too early for the group result.
- B. Correct.
- C. FROM identifies sources.
- D. DISTINCT removes duplicate outputs.

Rule to remember: Aggregate condition -> HAVING.

Practice Item 24

Which statement about HAVING is true?

- A. It is required whenever GROUP BY is used.
- B. It can restrict groups produced by GROUP BY.
- C. It filters source rows before grouping.
- D. It always removes at least one group.

Correct answer: B

Why

HAVING can remove groups after grouping.

Why the alternatives differ

- A. GROUP BY can exist without HAVING.
- B. Correct.
- C. WHERE filters source rows.
- D. The condition may allow all groups.

Rule to remember: Can is different from always.

Practice Item 25

`COUNT(DISTINCT department_id)` counts:

- A. All rows including NULL department IDs
- B. Distinct non-NULL department IDs
- C. Distinct complete employee rows
- D. Only duplicate department IDs

Correct answer: B

Why

COUNT(column) ignores NULL, and DISTINCT removes repeated values before counting.

Why the alternatives differ

- A. NULL is not counted.
- B. Correct.
- C. Only the argument is considered.
- D. Unique values are counted, not only duplicates.

Rule to remember: Apply COUNT and DISTINCT rules together.

First-Half Comparison Matrix

Often confused	Correct distinction	Typical wrong choice
WHERE vs HAVING	WHERE filters rows before groups; HAVING filters groups after aggregation	AVG(salary) in WHERE
DISTINCT vs GROUP BY	DISTINCT removes duplicate output rows; GROUP BY forms aggregation groups	Claiming they are always interchangeable
COUNT(*) vs COUNT(col)	All rows versus non-NULL column values	Counting NULL as a populated value
NVL vs COALESCE	Two-argument replacement versus first non-NULL among many	Using NULLIF as a fallback list
NULLIF vs NVL	NULL when equal versus replacement when NULL	Assuming both return zero
INSTR vs SUBSTR	Position versus extracted text	Returning text from INSTR
ROUND vs TRUNC	Rounding versus cutting off	Using same result when next digit is 5+
DATE+n vs ADD_MONTHS	Days versus calendar months	Treating 30 days as every month
& vs &&	Use/prompt versus define and reuse	Treating them as bind variables
ASC/DESC NULL defaults	ASC NULLS LAST; DESC NULLS FIRST	Assuming NULL is always last
Alias in ORDER BY vs WHERE	Usually usable in ORDER BY, not same-block WHERE	Filtering with a just-created alias
BETWEEN vs IN	Inclusive range versus listed values	Treating BETWEEN endpoints only

Final First-Half Study Checklist

- Explain logical versus physical database concepts without mixing constraints and indexes.
- Map ERD entities, attributes, keys, and relationships to SQL tables, columns, constraints, and joins.
- Predict aliases, concatenated strings, DISTINCT combinations, arithmetic precedence, and NULL results.
- Evaluate comparison predicates, logical precedence, row limiting, substitution variables, and sorting with NULLs.
- Calculate character, numeric, and date function results manually.
- Choose explicit conversions and compare NVL, NULLIF, COALESCE, and CASE.
- Compute aggregate results with NULL, validate GROUP BY select lists, and separate WHERE from HAVING.

Next-half boundary: The remaining syllabus starts with Displaying Data from Multiple Tables,

then subqueries, set operators, DML, indexes/synonyms/sequences, DDL, views, access control, dictionary views, and time zones. Those are intentionally excluded from this first-half PDF.

Sources and Scope

This reviewer is aligned to the topic list supplied in the screenshots and uses Oracle Database SQL terminology. Technical rules were checked against Oracle Database 19c SQL Language Reference sections for SELECT, SQL elements, functions, and aggregate behavior. SQL*Plus commands are client commands and can vary by client implementation.

<https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/index.html>

<https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/SELECT.html>

<https://docs.oracle.com/en/database/oracle/oracle-database/23/sqlrf/Aggregate-Functions.html>